```
itlab::Database::Writer.write(
  Gitlab::Database::Model::MyTable,
  [{ id: 1, foo: 'bar' }],
)
```

We intend to keep Gitlab::Database::Writer.backend to be as close to the backend-specific client implementation as possible. Having a wrapper around a vanilla client helps us address peripheral concerns such as service-discovery for the backends while still allowing the user to leverage features of a given client.

Iterations

Considering the large scope of this undertaking and the need for feedback around actual usage, we intend to build the proposed abstraction(s) across multiple iterations which can be described as follows:

Iteration 1 - Develop write abstraction with sync mode enabled

First, research and develop a simple write abstraction that our users can begin to use to write data into ClickHouse. This ensures our choice of the underlying client is well-researched and suffices to fulfill needs of as many reported use-cases as possible. Being able to see this running would help gather user-feedback and improve the write APIs/interfaces accordingly.

Given this feedback and more development with how we aim to deploy ClickHouse across our environments, it'd then be prudent to build into this abstraction necessary conventions, best practices and abstract away details around connection-pooling, service-discovery, etc.

Iteration 2 - Add support for schemas & data validation

In the next iteration, we plan to add support for schema usage and validation. This helps keep model definitions sane and allows for validating data to be inserted.

Iteration 3 - Add support for async mode, PoC with one backend

With the above two iterations well-executed, we can start to scale up our write abstractions adding the support for writing data into intermediate data stores before writing it into ClickHouse asynchronously. We aim to prototype such an implementation with atleast one such backend.

Further iterations

With a backend-agnostic abstraction becoming the ingestion interface a client interacts with, there's various other use-cases that can be solved from within this abstraction. Some of them are:

- Zero-configuration data ingestion from multiple sources
- Dynamically enriching data from multiple sources
- Offloading data to long-term retention data stores

Possible backend implementations

- Applications writing directly to ClickHouse
  - Application-local in-memory queueing/batching of data
  - Application-local persistent queueing/batching of data
- Non-local queueing/batching of data before eventually writing into ClickHouse
  - Managed cloud backends:
    - Google PubSub
    - AWS Kinesis
  - Self-managed backends:
    - CHProxy
    - Kafka
    - RedPanda
    - Vector
    - RabbitMQ

Additional complexity when using a non-local backend

- The need for running an additional process/sub-system that reads data from the concerned backend and writes it into ClickHouse efficiently and reliably.
- The additional hop across the backend also means that there might be potential delays in how soon this data lands into ClickHouse.

Though the points above describe additional complexity for an application, they can be treated as valid trade-off(s) assuming their need for data ingestion at scale.

Comparing backends across multiple dimensions

Dimension	CHProxy	Redis	Google PubSub	Apache Kafka
Operations	Trivial	Trivial	Managed	Non-trivial, complex
Data Retention	Non-durable	Non-durable	Durable	Durable
Performance	Good	Good	High	High
Data Streaming	None	Minimal	Good	Best
Suitable for self-managed environments	Trivial	Trivial	-	Complex

References

- ClickHouse use-cases within Manage
- List down all possible options for postgres to snowflake pipeline
- Design Spike for Snowplow For Data Event capture
- Audit Events Performance Limits

SECTION: engineering/architecture/design-documents/clickhouse_read_abstraction_layer/_index.md

{{< engineering/design-document-header >}}

Table of Contents

Summary

Provide a solution standardizing read access to ClickHouse or its alternatives for GitLab installations that will not opt-in to install ClickHouse. After analyzing different open-source tools and weighing them against an option to build a solution internally. The current recommended approach proposes to use dedicated database-level drivers to connect to each data source. Additionally, it proposes the usage of repository pattern to confine optionally database availability complexity to a single application layer.

Motivation

ClickHouse requires significant resources to be run, and smaller installations of GitLab might not get a return from investment with provided performance improvement. That creates a risk that ClickHouse might not be globally available for all installations and features might need to alternate between different data stores available. Out of all present & future ClickHouse use cases that have been already proposed as part of the working group 7 out of 10 uses data stores different than ClickHouse. Considering that context it is important to support those use cases in their effort to adopt ClickHouse by providing them with tools and guidelines that will standardize interactions with available data stores.

The proposed solution can take different forms from stand-alone tooling offering a unified interface for interactions with underlying data stores, to a set of libraries supporting each of the data stored individually backed by implementation guidelines that will describe rules and limitations placed around data stores interactions, and drawing borders of encapsulation.

Goals

- Limit the impact of optionally available data stores on the overall GitLab application codebase to single abstraction layer
- Support all data store specific features
- Support communication for satellite services of the main GitLab application

Non-goals

- This proposal does not directly consider write communication with database, as this is a subject of complementary effort
- This proposal does not directly consider schema changes and data migration challenges

Despite above points being non goals, it is acknowledge that they might impose some alterations to final solution which is expressed at the end of this document in the Open questions section.

Possible Solutions

High-level goals described in the previous paragraph can be achieved by both in-house-built solutions as well as by adopting open-source tools.
The following sections will take a closer look into both of those avenues

Recommended approach

In the spirit of MVC and iteration, it is proposed to start with a solution that would rely on drivers that directly interact
with corresponding data stores, like ActiveRecord for Ruby. For this solution to be able to achieve goals set for
this exit criteria and help mitigate the issue listed in the Motivation section of this document, such drivers need to be supported
by a set of development guidelines enforced with static code analysis.

Such a solution was selected as preferred upon receiving feedback from different members of the working group concerned
about the risk of limitations that might be imposed by open-source tools, preventing groups from taking advantage of ClickHouse
features to their fullest. Members collaborating around working group criteria presented in this document, agree that
concerns around limitations could be mitigated by building a comprehensive set of prototypes, however time and effort
required to achieve that surpass the limits of this working group. It is also important to notice that ClickHouse adoption
is in an exploratory stage, and groups might not being even able to state what are their requirements just yet.

Proposed drivers

Following ClickHouse documentation there are the following drivers for Ruby and Go

Ruby

1. ClickHouse Ruby driver - Previously selected for use in GitLab as part of the Observability group's research (see: issue)
1. Clickhouse::Activerecord

Go

1. ClickHouse/clickhouse-go - Official SQL database client.
1. uptrace/go-clickhouse - Alternative client.

Proposed client architecture

To keep the codebase well organized and limit coupling to any specific database engine it is

important to encapsulate
interactions, including querying data to a single application layer, that would present its interface to layers above in
similar vain to ActiveRecord interface propagation through abstraction layers

Keeping underlying database engines encapsulated makes the recommended solution a good two-way door decision that
keeps the opportunity to introduce other tools later on, while giving groups time to explore and understand their use cases.

At the lowest abstraction layer, it can be expected that there will be a family of classes directly interacting with the ClickHouse driver, those classes
following MVC pattern implemented by Rails should be classified as Models.

Models-level abstraction builds well into existing patterns and guidelines but unfortunately does not solve the challenge of the optional availability of the ClickHouse database engine for self-managed instances. It is required to design a dedicated entity that will house responsibility of selecting best database to serve business logic request.
From the already mentioned existing abstraction guidelines Finders seems to be the closest to the given requirements, due to the fact that Finders encapsulate database specific interaction behind their own public API, hiding database vendors detail from all layers above them.

However, they are closely coupled to ActiveRecord ORM framework, and are bound by existing GitLab convention to return ActiveRecord::Relation objects, that might be used to compose even more complex queries. That coupling makes Finders unfit to deal with the optional availability of ClickHouse because returned data might come from two different databases, and might not be compatible with each other.

With all that above in mind it might be worth considering adding a new entity into the codebase that would exist on a similar level of abstraction as Finders yet it would be required to return an Array of data objects instead.

Required level of isolation can be achieved with usage of a repository pattern. The repository pattern is designed to separates business / domain logic from data access concerns, which is exactly what this proposal is looking for.
What is more the repository pattern does not limits operations performed on underlying databases allowing for full utilization of their features.

To implement the repository pattern following things needs to be created:

1. A strategy for each of supported databases, for example: MyAwesomeFeature::Repository::Strategies::ClickHouseStrategy and MyAwesomeFeature::Repository::Strategies::PostgreSQLStrategy. Strategies are responsible for implementing communication with underlying database ie: composing queries
1. A repository that is responsible for exposing high level interface to interact with database using one of available strategies selected with some predefined criteria ie: database availability. Strategies used by single repository must share the same public interface so they can be used interchangeable
1. A Plain Old Ruby Object(PORO) Model that represents data in business logic implemented by application layers using repository. It have to be database agnostic

It is important to notice that the repository pattern based solution has already been implemented by Observability group (kudos to: @ahegyi, @splattael and @brodock). ErrorTracking::ErrorRepository is being used to support migration of error tracking features from PostgreSQL to ClickHouse (integrated via API), and uses feature flag toggle as database selection criteria, that is great example of optional availability of database.

ErrorRepository is using two strategies:

1. OpenApiStrategy to interact with ClickHouse using API proxy entity
1. ActiveRecordStrategy to interact with PostgreSQL using ActiveRecord framework

Each of those strategies return data back to abstraction layers above using following PORO Models:

1. Gitlab::ErrorTracking::Error
1. Gitlab::ErrorTracking::DetailedError

Additionally ErrorRepository is great example of remarkable flexibility offered by the repository pattern in terms of supported types of data stores, allowing to integrate solutions as different as a library and external service API under single unified interface. That example presents opportunity that the repository pattern in the future might be expanded beyond needs of ClickHouse and PostgreSQL when some use case would call for it.

Following merge request documents changes done by observability group in order to migrate from using current GitLab architecture based on ActiveRecord Models, Services and Finders to the repository pattern.

Possible ways to enforce client architecture

It is not enough to propose a client-side architecture for it to fully be established as common practice it needs
to be automatically enforced, reducing the risk of developers unconsciously going against it. There are multiple ways to
introduce automated verification of repository pattern implementation including:

1. Utilize ActiveRecord query subscribers in a similar way toDatabase::PreventCrossJoins in order to detect queries to ClickHouse executed outside of Strategies
1. Expanding CodeReuse RuboCop rules to flag all usage of ClickHouse driver outside of Strategies
1. Create RuboCop rule that detects calls to utility method that checks the presence of ClickHouse instance (ie: CurrentSettings.clickhouseenabled?) that are being made outside of Repositories

At this development stage, authors see all of the listed options as viable and promising, therefore a decision about which ones to use would be deferred to the moment when the first repository pattern implementation for ClickHouse will emerge.

Overview of open-source tools

In this section authors provide an overview of existing 3rd party open-source solutions that

were considered as alternative approaches to achieve stated goal, but was not selected as recommended approach.

Evaluation criteria

1. License (MUST HAVE)

1. Solutions must be open source under an acceptable license.

2. Support for different data stores (MUST HAVE)

1. It focuses on the fact whether the proposed abstraction layer can support both ClickHouse and PostgreSQL (must have)
1. Additional consideration might be if more than the two must-have storages are supported
1. The solution must support the minimum required versions for PostgreSQL

3. Protocol compatibility

Every abstraction layer comes at the cost of limited API compared to direct access to the tool. This exit criterion is trying to bring understanding to the degree of trade-off being made on limiting tools API for the sake of a common abstraction.

1. List what read operations can be done via PostgreSQL and ClickHouse (selects, joins, group by, order by, union etc)
1. List what operations can be done with the proposed abstraction layer, how complicated it is to do such operations, and whether are there any performance concerns when compared to running operations natively
1. Does it still allow for direct access to a data source in case the required operation is not supported by the abstraction layer, eg: ActiveRecord allows for raw SQL strings to be run with execute

4. Operational effort

1. Deployment process: how complex is it? Is the proposed tool a library tool that is being added into the stack, or does it require additional services to be deployed independently along the GitLab system. What deployment types does the tool support (Kubernetes/VMs, SaaS/self-managed, supported OS, cloud providers). Does it support offline installation.
1. How many hardware resources does it need to operate
1. Does it require complex monitoring and operations to assure stable and performant services
1. Matured maintenance process and documentation around it: upgrades, backup and restore, scaling
1. High-availability support. Does the tool have documentation how to build HA cluster and perform failovers for self-managed? Does the tool support zero-downtime upgrade?
1. FIPS and FedRAMP compliance
1. Replication process and how the new tool would fit in GitLab Geo.

5. Developer experience

1. Solutions must have well-structured, clear, and thoroughly documented APIs to ease adoption and reduce the learning curve.

6. Maturity (nice to have)

1. How long does the solution exist? Is it used often? Does it have a stable community? If the license permits forking tool is also a considerable option

7. Tech fit

1. Is the solution written in one of the programming languages we use at GitLab so that we can more easily contribute with bug fixes and new features?

8. Interoperability (Must have)

1. Can the solution support both the main GitLab application written in Ruby on Rails also satellite services like container registry that might be written in Go

Open - Source solutions

1. Cube.dev

Cube.dev

Evaluation

1. License
   Apache 2.0 + MIT ·
1. Support for different data stores
   Yes ·
1. Protocol compatibility
   It uses OLAP theory concepts to aggregate data. This might be useful in some use cases like aggregating usage metrics, but not in others. It has APIs for both SQL queries and their own query format.
1. Operational effort
   Separate service to be deployed using Docker or k8s. Uses Redis as a cache and data structure store.
1. Developer experience
   Good documentation
1. Maturity
   Headless BI tools themselves are a fairly new idea, but Cube.js seems to be the leading open-source solution in this space.
   The Analytics section uses it internally for our Product Analytics stack.
1. Tech fit
   Uses REST and GraphQL APIs. It has its own query and data schema formats, but they are well-documented. Data definitions in either YAML or JavaScript.

Comment

The solution is already being used as a read interface for ClickHouse by ~"group::product analytics",
to gather first hand experience there was a conversation held with @mwoolf with key conclusions being:

1. ClickHouse driver for cube.dev is community-sourced, and it does not have a maintainer as of now, which means there is no active development. It is a small and rather simple repository that should work at least until a new major version of ClickHouse will arrive with some breaking changes
1. Cube.dev is written in Type Script and JavaScript which are part of GitLab technical stack, and there are engineers here with expertise in them, however Cube.dev is expected to be mostly used by backend developers, which does not have that much experience in mentioned technologies
1. Abstraction layer for simple SQL works, based on JSON will build correct query depending on the backend
1. Data store-specific functions (like window funnel ClickHouse) are not being translated to other engines, which requires additional cube schemas to be built to represent the same data.
1. Performance so far was not an issue both on local dev and on AWS VPS millions of rows import load testing
1. It expose postgres SQL like interface for most engines, but not for ClickHouse unfortunately so for sake of working group use case JSON API might be more feasible
1. Cube.dev can automatically generate schemas on the fly, which can be used conditionally in the runtime handling optional components like ClickHouse

There is also a recording of that conversation available.

2. ClickHouse FDW

ClickHouse FDW

Evaluation

A ClickHouse Foreign Data Wrapper for PostgreSQL. It allows ClickHouse tables to be queried as if they were stored in PostgreSQL.
Could be a viable option to easily introduce ClickHouse as a drop-in replacement when Postgres stops scaling.

1. License
   Apache 2.0 ·
1. Support for different data stores
   Yes, by calling ClickHouse through a PostgreSQL instance. ·
1. Protocol compatibility
   Supports SELECT, INSERT statements at a first glance. Not sure about joins. Allows for raw SQL by definition.
1. Operational effort
   1. A PostgreSQL extension. Requires some mapping between the two DBs.
   1. Might have adversary impact on PostgreSQL performance, when execution would wait for response from ClickHouse waisting CPU cycles on waiting
   1. Require exposing and managing connection between deployments of PostgreSQL and ClickHouse
1. Developer experience
   TBD
1. Maturity
   It's been around for a few years and is listed in ClickHouse docs, but doesn't seem to be widely used.
1. Tech fit
   Raw SQL statements.

Comment

3. Clickhouse::Activerecord

Clickhouse::Activerecord

Evaluation

1. License
   MIT License ·
1. Support for different data stores
   Yes, in the sense that it provides a Clickhouse adapter for ActiveRecord in the application layer so that it can be used to query along PostgreSQL. ·
1. Protocol compatibility
   Not sure about joins - no examples.
1. Operational effort
   Ruby on Rails library tool - ORM interface in a form of an ActiveRecord adapter.
1. Developer experience
   Easy to work with for developers familiar with Rails.
1. Maturity
   Has been around for a few years, but repo activity is scarce (not a bad thing by itself, however).
1. Tech fit
   Rails library, so yes.

Comment

4. Metriql

Metriql

Evaluation

A headless BI solution using DBT to source data. Similar to Cube.dev in terms of defining metrics from data and transforming them with aggregations.
The authors explain the differences between Metriql and other BI tools like Cube.js in this FAQ entry.

1. License
   Apache 2.0 ·
1. Support for different data stores
   Uses DBT to read from data sources, so CH and PostgreSQL are possible.
1. Protocol compatibility
   It uses OLAP theory concepts to aggregate data. It does allow for impromptu SQL queries through a REST API.
1. Operational effort
   It's a separate service to deploy and requires DBT.
1. Developer experience
   I assume it requires DBT knowledge to set up and use. It has a fairly simple REST API documented here.

1. Maturity
   First release May 2021, but repo activity is scarce (not a bad thing by itself).
1. Tech fit
   Connects with BI tools through a REST API or JDBC Adapter. Allows querying using SQL or MQL (which is a SQL flavor/subset).

Comment

5. Notable rejected 3rd party solutions

ETL only solutions like Airflow and Meltano, as well as visualization tools like Tableau and Apache Superset, were excluded from the prospect list as they are usually clearly outside our criteria.

pg2ch
PostgreSQL to ClickHouse mirroring using logical replication.
Repo archived; explicitly labeled not for production use. Logical replication might not be performant enough at our scale - we don't use it in our PostgreSQL DBs because of performance concerns.

Looker
BI tooling.
Closed-source; proprietary.

Hasura
GraphQL interface for database sources.
No ClickHouse support yet.

dbt Server
HTTP API for dbt. MariaDB Business Source License (BSL) ·

Open questions

1. This proposal main focus is read interface, however depending on outcome of complementary effort that focus on write interface similar concerns around optional availability might be applicable to write interaction. In case if ingestion pipeline would not resolve optional availability challenges for write interface it might be considerable to include write interactions into repository pattern implementation proposed in this document.
1. Concerns around ClickHouse schema changes and data migrations is not covered by any existing working group criteria, even though solving this challenges as a whole is outside of the scope of this document it is prudent to raise awareness that some alterations to proposed repository pattern based implementation might be required in order to support schema changes.

SECTION: engineering/architecture/design-documents/clickhouse_usage/_index.md

{{< engineering/design-document-header >}}

Summary

ClickHouse is an open-source column-oriented database management system. It can efficiently filter, aggregate, and sum across large numbers of rows. In FY23, GitLab selected ClickHouse as its standard data store for features with big data and insert-heavy requirements such as Observability and Analytics. This blueprint is a product of the ClickHouse working group. It serves as a high-level blueprint to ClickHouse adoption at GitLab and references other blueprints addressing specific ClickHouse-related technical challenges.

Motivation

In FY23-Q2, the Monitor:Observability team developed and shipped a ClickHouse data platform to store and query data for Error Tracking and other observability features. Other teams have also begun to incorporate ClickHouse into their current or planned architectures. Given the growing interest in ClickHouse across product development teams, it is important to have a cohesive strategy for developing features using ClickHouse. This will allow teams to more efficiently leverage ClickHouse and ensure that we can maintain and support this functionality effectively for SaaS and self-managed customers.

Use Cases

Many product teams at GitLab are considering ClickHouse when developing new features and to improve performance of existing features.

During the start of the ClickHouse working group, we documented existing and potential use cases and found that there was interest in ClickHouse from teams across all DevSecOps stage groups.

Goals

As ClickHouse has already been selected for use at GitLab, our main goal now is to ensure successful adoption of ClickHouse across GitLab. It is helpful to break down this goal according to the different phases of the product development workflow.

1. Plan: Make it easy for development teams to understand if ClickHouse is the right fit for their feature.
1. Develop and Test: Give teams the best practices and frameworks to develop ClickHouse-backed features.
1. Launch: Support ClickHouse-backed features for SaaS and self-managed.
1. Improve: Successfully scale our usage of ClickHouse.

Non-goals

A strategy for integrating ClickHouse into GitLab Dedicated has not begun. Leadership guidance has been to wait until there is clearer demand for ClickHouse backed features before prioritizing this.

Product roadmap

FY24 H2 (past)

In FY24 Q2 we began working to integrate ClickHouse with GitLab.com to support multiple

features under development (see issue). We did not move forward attempting to integrate with self managed at this time due to the uncertain costs and management requirements for self-managed instances. This near-term implementation will be used to develop best practices and strategy to direct self-managed users. This will also constantly shape our recommendations for self-managed instances that want to onboard ClickHouse early. As of FY24 Q3 ClickHouse is available for use with GitLab.com.

FY25 H1 (current)

After we have formulated best practices of managing ClickHouse ourselves for GitLab.com, we will begin to offer supported recommendations for self-managed instances that want to run ClickHouse themselves. During this phase we will allow users to "Bring your own ClickHouse" similar to our approach for Elasticsearch. For the features that require ClickHouse for optimal usage (Value Streams Dashboard, Product Analytics), this will be the initial go-to-market action. Notably, the Observability team has made the decision to support self-managed users via GitLab Cloud Connector instead of following this approach.

Long-term

We will work towards a packaged reference version of ClickHouse capable of being easily managed with minimal cost increases for self-managed users. We should be able to reliably instruct users on the management of ClickHouse and provide accurate costs for usage. This will mean any feature could depend on ClickHouse without decreasing end-user exposure.

Best Practices

Best practices and guidelines for developing performant, secure, and scalable features using ClickHouse are located in the ClickHouse developer documentation.

Cost and maintenance analysis

ClickHouse components cost and maintenance analysis is located in the ClickHouse Self-Managed component costs and maintenance requirements.

SECTION: engineering/architecture/design-documents/clickhouse_usage/self_managed_costs_and_requirements.md

Summary

ClickHouse requires additional cost and maintenance for self-managed customers:

- Resource allocation cost: ClickHouse requires a considerable amount of resources to run optimally.
  - Minimum cost estimation shows that setting up ClickHouse can be applicable only for very large Reference Architectures: 25k and up.
- High availability: ClickHouse SaaS supports HA. No documented HA configuration for self-managed at the moment.
- Geo setups: Sync and replication complexity for GitLab Geo setups.

- Upgrades: An additional database to maintain and upgrade along with existing Postgres database. This also includes compatibility issues of mapping GitLab version to ClickHouse version and keeping them up-to-date.
- Backup and restore: Self-managed customers need to have an engineer who is familiar with backup strategies and disaster recovery process in ClickHouse or switch to ClickHouse SaaS.
- Monitoring: ClickHouse can use Prometheus, additional component to monitor and troubleshoot.
- Limitations: Azure object storage is not supported. GitLab does not have the documentation or support expertise to assist customers with deployment and operation of self-managed ClickHouse.
- ClickHouse SaaS: Customers using a self-managed GitLab instance with regulatory or compliance requirements, or latency concerns likely cannot use ClickHouse SaaS.

Minimum self-managed component costs

Based on ClickHouse spec requirements analysis
and collaborating with ClickHouse team, we identified the following minimal configurations for ClickHouse self-managed:

1. ClickHouse High Availability (HA)
   - ClickHouse - 2 machines with >=16-cores, >=64 GB RAM, SSD, 10 GB Internet. Each machine also runs Keeper.
   - Keeper - 1 machine with 2 CPU, 4 GB of RAM, SSD with high IOPS
1. ClickHouse non-HA
   - ClickHouse - 1 machine with >=16-cores, >=64 GB RAM, SSD, 10 GB Internet.

The following cost table was compiled using the machine CPU and memory requirements for ClickHouse, and comparing them to the
GitLab Reference Architecture sizes and costs from the GCP calculator.

Reference Architecture	ClickHouse type	ClickHouse cost / (GitLab cost + ClickHouse cost)
20 RPS / 1k users - non HA	non-HA	78.01%
40 RPS / 2k users- non HA	non-HA	44.50%
60 RPS / 3k users - HA	HA	37.87%
100 RPS / 5k users - HA	HA	30.92%
200 RPS / 10k users - HA	HA	20.47%
500 RPS / 25k users - HA	HA	14.30%
1000 RPS / 50k users - HA	HA	8.16%

NOTE:
The ClickHouse Self-Managed component evaluation is the minimum estimation for the costs with a simplified architecture.

The following components increase the cost, and were not considered in the minimum calculation:

- Disk size - depends on data size, hard to estimate.
- Disk types - ClickHouse recommends fast SSDs.
- Network usage - ClickHouse recommends using 10 GB network, if possible.
- For HA we sum minimum cost across all reference architectures from 60 RPS / 3k users to 1000 RPS / 50k users, but HA specs tend to increase with user count.

Resources

- Research and understand component costs and maintenance requirements of running a ClickHouse instance with GitLab
- ClickHouse for Error Tracking on GitLab.com

SECTION: engineering/architecture/design-documents/cloudflare-standardization/_index.md

<!-- Design Documents often contain forward-looking statements -->
<!-- vale gitlab.FutureTense = NO -->

{{< engineering/design-document-header >}}

Summary

This document describes the architecture and implementation approach for a set of Terraform modules that provide a secure and extensible interface to Cloudflare configuration across GitLab's internal teams. As GitLab's edge networking needs grow and more teams adopt Cloudflare, we need a standardized approach that enables teams to implement secure and compliant edge networking solutions achieving operational excellence.

The proposed solution delivers a set of Terraform modules with sensible defaults, documentation, and clear upgrade paths. This standardization will enable teams to use Cloudflare's capabilities effectively, reducing implementation complexity across the organization.

This work is being tracked in this epic,
where we discuss the prioritization and project management further.

Motivation

GitLab's usage of Cloudflare as our preferred edge networking provider continues to grow, with several teams implementing solutions for DNS management, WAF configuration, and worker deployments across our infrastructure estate (
example 1,
example 2,
cloudflare-waf-rules module,
dns-record module
).
This creates an opportunity to establish a more unified and scalable foundation that can benefit all teams.

As we move toward a maturing GitLab platform, with standardized infrastructure offerings and a platform-first mindset, establishing patterns for Cloudflare management will provide significant strategic value by reducing implementation complexity and standardizing our security and compliance at the edge. Through testing, documentation, and clear upgrade processes, we can enable teams to

implement and maintain their Cloudflare configurations independently. By iterating on the approach taken by the
cloudflare-waf-rules
module for WAF rules and rate limit configuration we can strengthen our ability to respond to security and compliance risks as they emerge, and allow teams to implement Cloudflare functionality in a scalable way.

This approach also positions us well for upcoming maintenance work, including upgrading the Cloudflare provider, allowing us to implement a coordinated, reduced-risk upgrade process across all implementations.

Goals

- Provide an Edge/WAF abstraction suitable for GitLab's requirements, offering a well-defined and easy-to-understand interface, encouraging encapsulation, modularization and replaceability of implementation.
- Establish consistent security and compliance standards across all Cloudflare implementations
- Deliver flexible and extensible modular configurations that accommodate both common use cases and specialized requirements
- Accelerate team velocity by providing well-documented, tested infrastructure patterns
- Establish a foundation for efficient upgrades and security improvements across all implementations
- Centralized documentation for internal customers and operator audiences that enables self-serve implementation and support with high levels of autonomy
- A defined process for interaction with the maintainer team for support beyond existing documentation

Non-Goals

- Creating team-specific implementations; instead, we will provide the foundational modules that teams can use to build their own solutions
- Managing the day-to-day operations of each team's specific Cloudflare configurations
  - This includes configuration changes and version upgrades of the modules
- Replicating Cloudflare functionality within GitLab's product offerings

Proposal

We propose developing a set of Terraform modules with 3 key themes:

1. A common entrypoint module built as the primary interface with sensible defaults
1. Specialized modules implementing specific subsets of Cloudflare functionality. This includes DNS and WAF.
1. Data-only submodules providing standardized configuration patterns that can be reused across implementations

The common entrypoint will be the default choice for implementers to add Cloudflare configuration to their applications. Internally this will use the

specialized modules to allow for a path to greater implementation flexibility if required.

All modules will be defined with sane defaults that covers the majority of cases, with interfaces designed to allow extensibility. This will be driven by the focus on the data-only submodules which will provide extensible default configurations composed of a set of common case configurations.

This allows teams to independently enable Cloudflare functionality for their use-case with minimal configuration, scaling through to more specialized module use where required.This flexibility ensures that both common use cases and complex requirements are well-supported.

By creating a centralized set of modules we can codify Cloudflare expertise and support team autonomy, creating a sustainable platform for edge networking across GitLab.

Security will be a priority in our design, with pre-configured security settings aligned with GitLab's requirements built into the modules.
This includes WAF rule sets optimized for common GitLab application patterns and rate limiting configurations to prevent abuse. By establishing secure defaults, we ensure that all Cloudflare implementations maintain a baseline level of security.

The modules will follow existing infrastructure-as-code principles and practices that we practice at GitLab, using Terraform as our preferred IaC platform, with consistent interfaces, testing, and extensive documentation.

Success Metrics

The effectiveness of this initiative will be demonstrated through several key indicators of organizational capability and team empowerment:

- Successful implementations of Cloudflare solutions independently
- Team satisfaction and confidence in using our Cloudflare modules, measured through feedback form collection and qualitative feedback sessions
- Time teams spend updating and upgrading their configuration
- Adoption count of the Cloudflare modules
- No increase in S2 and S1 incident rates related to Cloudflare configurations

Module Development Principles

- Utilize leaky abstractions to build on top of Cloudflare's existing API structure
  - This allows us to build on top of Cloudflare's existing patterns and documentation
- Backwards compatibility as a top priority
  - Strict versioning using semantic versioning principles
  - Deprecation notices with clear migration paths for disruptive changes to the interface or underlying resources
  - For any major version change, we MUST provide upgrade documentation for any manual interventions required for implementers.

  - Automated testing for common use cases to detect breaking changes
- Self-service focus where implementers can follow documentation to implement autonomously
- Consistent interfaces across modules
  - Maps, objects, and lists will be the primary interface to modules
    - These are generally more flexible than scalar alternatives and avoid the need for duplicate updates across modules when new options are added

Design and implementation details

Beneath the root module, we will implement several standalone modules that specialize in a specific Cloudflare functionality area. Examples include cloudflare/dns for DNS configuration, and cloudflare/waf for Web Application Firewall configuration.

When we observe common configuration patterns emerging across implementations, we will document these use cases, and for very common instances we will develop generic use-case modules. These modules will be based on the core cloudflare module and may implement patterns such as simple DNS configuration or Worker implementations.

To support customization without sacrificing standardization, we will create reusable configuration options in {module-path}/data/ sub-modules. An example is WAF rules, where sets of WAF rules may be common across many instances but are not appropriate for all consumers. Where we are building distinct sets of configuration, we will also build a default configuration for ease of use and extensibility. For example, where we have multiple WAF rulesets defined.

Module Relationships

```
mermaid
flowchart TD
    %% Individual teams at the top
    user1["· DNS Team<br/>Simple Setup"]
    user2["·· Security Team<br/>Custom WAF Rules"]
    user3["·· Platform Team<br/>Advanced Multi-Service"]
    %% Main entry point module
    subgraph entry-module[" · Entry Point Module "]
        cloudflare[cloudflare]
        cf-data[cloudflare/data]
        cloudflare --> cf-data
    end
    %% Functional modules layer
    subgraph functional-layer[" · Functional Modules "]
        direction LR
        cf-dns[cloudflare/dns]
        cf-workers[cloudflare/workers]
        cf-logging[cloudflare/logging]
        cf-waf[cloudflare/waf]
    end
    %% Detailed WAF module (separate for clarity)
```

```
    subgraph waf-detail[" · WAF Module Details "]
        direction TB
        waf-data[cloudflare/waf/data]
        subgraph waf-rulesets[" · Rulesets "]
            direction LR
            waf-default[default]
            waf-gcs[gcs]
            waf-bots[bots]
            waf-default --> waf-gcs
            waf-default --> waf-bots
        end
        waf-data --> waf-rulesets
    end
    %% User to entry point connections
    user1 --> cloudflare
    user2 --> cloudflare
    user2 --> cf-data
    user3 --> cf-waf
    user3 --> waf-data
    %% Entry point to functional modules
    cloudflare --> functional-layer
    %% Data flow connections
    cf-data --> waf-data
    cf-waf -.-> waf-data
```

Example Terraform configurations

Entrypoint module illustration

This example shows a minimal usage of the cloudflare common entrypoint module.

Without overrides for additional options, this will configure the zone in
Cloudflare, create a DNS A record, and configure default rulesets for WAF
custom rules and rate limit rules.

```
terraform
module "cloudflare" {
    source = "path/to/entrypoint/module"

    zone = {
        root = "example.gitlab.com"
        plan = "free"
    }

    dns = {
        a = {
             Create an A record for test.example.gitlab.com
            test = {
                name    = "test"
```

```
            records = [
              "127.0.0.1"
            ]
          }
        }
    }
}
```

This next example illustrates a detailed example of multiple configurations through the common entrypoint module.

```
terraform
module "cloudflaredata" {
    source = "path/to/entrypoint/module//modules/data"

    zone = "example.gitlab.com"
}

module "cloudflare" {
    source = "path/to/entrypoint/module"

    zone = {
        root = "example.gitlab.com"
        plan = "free"
    }

    dns = {
        a = {
            Create an A record for test.example.gitlab.com
            test = {
                name    = "test"
                records = [
                  "127.0.0.1"
                ]
            }

            Create a proxied A record with a custom TTL
            testproxied = {
                name    = "test-proxied"
                records = [
                  "127.0.0.1"
                ]
                proxied = true
                ttl     = 300
            }
        }

        mx = {
            Create a test MX record
```

```
        test = {
          name     = "test"
          records  = [
            "mail.example.com"
          ]
          priority = 10
        }
      }
    }

    waf = {
      custom = {
        Build a custom ruleset from predefined rulesets

        Without explicit configuration this would default to
module.data.waf.custom.rulesets.default.
        This would provide the rules that we expect the majority of implementations to use
as a base.
        rules = concat(
          module.data.waf.custom.rulesets.gcs,
          module.data.waf.custom.rulesets.bots,
        )
      }
      ratelimits = {
        Disable rate limits by overriding with an empty list
        rules = []
      }
    }
}
```

Custom WAF rule implementation

This example illustrates the expected usage pattern for direct submodule usage, with a custom list of rules defined to override the default. This approach will typically be used when multiple explicit configuration stages are preferred.

zoneid and zone are provided here to allow us to modify the records, and generate rulesets that apply to the specified zone where required. When used through the common entrypoint this is automated through the zone top level variable.

```
terraform
module "wafrulesets" {
  source = "path/to/waf/module//modules/data"

  zone = "example.gitlab.com"
}

module "waf" {
```

```
  source = "path/to/waf/module"

  zoneid = "..."
  zone    = "example.gitlab.com"

  rules = concat(
    module.wafrulesets.bots,
    module.wafrulesets.gcs
  )
}
```

Testing Strategy

We will be using Terraform tests
to test every module. This will use a combination of unit-style tests with
mocked resources and end-to-end tests that create resources in a test zone
following patterns that we observe across implementation of the modules.

This allows us to test our computed values and created resources quickly during
development, and to ensure that we are providing well-tested golden paths that
our implementers use. This will aid our efforts to provide a stable interface
to our implementers, and allow us to ensure that any difference in resource
creation and configuration across module revisions are expected.

Documentation Strategy

Documentation is essential for the success of this initiative. We
will provide detailed README files for each module, explaining its purpose,
inputs, outputs, and example usage. For common use cases, we will create example
configurations that teams can use as starting points for their own
implementations. When major version changes occur, we will provide upgrade
guides that walk teams through the process of migrating to the new
version. Additionally, we will create internal knowledge base articles for
GitLab-specific implementations, addressing unique requirements and
considerations for our environment.

Additionally, each module will contain user and operator documentation on it's
usage. The common entrypoint will also provide an entrypoint for a
documentation site that can be hosted on GitLab Pages for internal customers.
This will source the documentation from each module and compile all relevant
documentation into a single source of truth for internal customers and operators
to use as a reference. This approach provides additional benefits, including
aligning documentation with changes made to modules. This ensures that we can
keep documentation updated alongside code changes made to the modules.

Initial development and migration

This implementation will focus on creating the common entrypoint first to allow
for early adoption, expanding the responsibilities as more functionality is

added through the specialized modules. This will allow us to gradually implement functionality and deliver impact faster. This is also an opportunity to validate that our upgrade process is defined and works across our modules.

As we add functionality we will document migration paths from any existing implementations that we are aware of for the same functionality. This will allow us to further iterate on our provided documentation and ensure it is fit for purpose.

Alternative Solutions

Continue with Custom Implementations Per Team

Continuing with the current approach of custom implementations per team would have several advantages and disadvantages:

Pros:

- Teams maintain complete control over their configurations
- No upfront investment required
- Faster implementation for teams with immediate needs

Cons:

- Inconsistent security and compliance practices
- Duplicate effort across teams
- Higher risk of configuration drift over time
- Growing maintenance support burden as more teams adopt Cloudflare

Fully Centralized Cloudflare Configuration

An alternative approach would be to fully centralize Cloudflare configuration management within a single repository.

Pros:

- Maximum standardization and control
- Consistent security posture
- Single point of maintenance
- Simplified compliance auditing

Cons:

- Reduced maintainability for teams with unique requirements
- Creates bottleneck for changes on the maintaining team
- Does not foster self-service culture
- May slow down teams with time-sensitive requirements
- Teams relying on Cloudflare functionality integration as part of a platform will require separate out-of-band changes to enable features for each Cloudflare-related change a tenant requires

Documentation-only improvements

We can reduce the initial implementation burden by creating
documentation for our current set of Cloudflare modules.

Pros:

- Reduces time to initial implementation
- Minimal implementation required
- Encourages a self-service culture for initial configuration

Cons:

- Manual configuration required for more complex implementations
- No unified maintenance support for teams
- Maintenance responsibility for Cloudflare configurations spread across teams
- Susceptible to drift as standard configurations change

SECTION: engineering/architecture/design-documents/cloud_connector/_index.md

{{< engineering/design-document-header >}}

Summary

The Cloud Connector team is now disbanded. These pages are kept for now to give historical context.

This design doc covers architectural decisions and proposed changes aligned with the team's technical vision.
Refer to the official architecture documentation
for an accurate description of the current status.

Motivation

Our "big problem to solve" is to bring feature parity to our SaaS and self-managed offerings.
Until now, SaaS and self-managed (SM) GitLab instances consume features only from the AI gateway,
which also implements an Access Layer to verify that a given request is allowed
to access the respective AI feature endpoint.
<!-- TODO: change to new design doc URL -->

This approach has served us well because it:

- Required minimal changes from an architectural standpoint to allow SM users to consume AI features hosted by us.
- Caused minimal friction with ongoing development on GitLab.com.
- Reduced time to market.

However, the AI gateway alone does not sufficiently abstract over a wider variety of features, as by definition it is designed to serve AI features only.

Goals

We will use this blueprint to make incremental changes to Cloud Connector's technical framework to enable other backend services to service self-managed/GitLab Dedicated customers in the same way
the AI gateway does today. This will directly support our mission of bringing feature parity to all GitLab customers.

The major areas we are focused on are:

- Provide single access point for customers.
  We found that customers are not keen on configuring their web proxies and firewalls
  to allow outbound traffic to an ever growing list of GitLab-hosted services. We therefore decided to
  install a global, load-balanced entry point at cloud.gitlab.com. This entry point can make simple
  routing decisions based on the requested path, which allows us to target different backend services
  as we broaden the feature scope covered by Cloud Connector.
  - Status: done. The decision was documented as ADR-001.
- Remove OIDC key discovery.
  The original architecture for Cloud Connector relied heavily on OIDC discovery to fetch JWT validation keys.
  OIDC discovery is prone to networking and caching problems and adds complexity to solve a problem we don't have.
  Our proposed alternative to OIDC discovery is to package the public keys used for token validation from our well-known token issuers with Cloud Connector backends directly instead of fetching them over the network.
  - Status: parked. We may publish a follow up ADR for an alternative approach. The decision was documented as ADR-002
- Rate-limiting features.
  During periods of elevated traffic, backends integrated with Cloud Connector such as
  AI gateway or TanuKey may experience resource constraints. GitLab should apply a consistent strategy when deciding which instance
  should be prioritized over others. This strategy should be uniform across all Cloud Connector services.
  - Status: In Progress.
- Extract CloudConnector unitprimitive configuration and logic
  We will implement a new unit primitive-based configuration system by extracting it to an external library (gitlab-cloud-connector) that will serve as the Single Source of Truth (SSoT).
  This library will be available as both a Ruby gem and a Python package. The decision was documented as ADR-003
  - Status: In Progress.

Decisions

- ADR-001: Use load balancer as single entry point

- ADR-002: Remove OIDC key discovery
- ADR-003: Centralize Unit Primitives configuration

SECTION: engineering/architecture/design-documents/cloud_connector/decisions/001_lb_entry_point.md

Context

The original iteration of the blueprint suggested to stand up a dedicated Cloud Connector edge service,
through which all traffic that uses features under the Cloud Connector umbrella would pass.

The primary reasons for why we wanted this to be a dedicated service were to:

1. Provide a single entry point for customers. We identified the ability for any GitLab instance
   around the world to consume Cloud Connector features through a single endpoint such as
   cloud.gitlab.com as a must-have property.
1. Have the ability to execute custom logic. There was a desire from product to create a space where we can
   run cross-cutting business logic such as application-level rate limiting, which is hard or impossible to
   do using a traditional load balancer such as HAProxy.

Decision

We decided to take a smaller incremental step toward having a "smart router" by focusing on the ability to provide a single endpoint through which Cloud Connector traffic enters our infrastructure. This can be accomplished using simpler means than deploying dedicated services, specifically
by pulling in a load balancing layer listening at cloud.gitlab.com that can also perform simple routing
tasks to forward traffic into feature backends.

Our reasons for this decision were:

1. Unclear requirements for custom logic to run. We are still exploring how and to what extent we would
   apply rate limiting logic at the Cloud Connector level. This is being explored in
   issue 429592. Because we need to have a single
   entry point by January, and because we think we will not be ready by then to implement such logic at the
   Cloud Connector level, a web service is not required yet.
1. New use cases found that are not suitable to run through a dedicated service. We started to work with
   the Observability group to see how we can bring the GitLab Observability Backend (GOB) to Cloud Connector
   customers in MR 131577.

In this discussion it became clear that due to the large amounts of traffic and data volume passing
through GOB each day, putting another service in front of this stack does not provide a sensible
risk/benefit trade-off. Instead, we will probably split traffic and make Cloud Connector components
available through other means for special cases like these (for example, through a Cloud Connector library).

We are exploring several options for load-balancing this new endpoint in issue 429818
and are working with the Infrastructure:Foundations team to deploy this in issue 24711.

Consequences

We have not yet discarded the plan to build a smart router eventually, either as a service or through other means, but have delayed this decision in face of uncertainty at both a product and technical level. We will reassess how to proceed in Q1 2024.

SECTION: engineering/architecture/design-documents/cloud_connector/decisions/002_remove_oidc_key_discovery.md

Context

We are exploring approaches to move away from OIDC discovery for Cloud Connector token validation for the following reasons:

- OIDC discovery is prone to networking and caching problems. If the endpoint or system is degraded or caches are stale, Cloud Connector backends cannot validate any AI requests anymore. With an increasing number of Cloud Connector backends, this issue is multiplied. An example is issue 480018 (internal for security reasons).
- OIDC adds complexity to solve a problem we don't have. It primarily solves the problem of 3rd parties that don't know or control each other to exchange identity and key information through a standardized web interface. This is not necessary for Cloud Connector, because all systems involved that dispense or authenticate tokens are built, operated and controlled by GitLab.
- OIDC discovery requires a callback to gitlab.com from all Cloud Connector backends. This means to support Cells, where customers can reside in different Cells, the application secret holding these keys must be managed either so as to be shared across all Cells, or sharded and the request routed accordingly for the backend to obtain the right key set. We can eliminate this problem entirely by simply not having gitlab.com publish these keys through OIDC endpoints and removing this callback. See issue 451149 for more information.

Our proposed alternative to OIDC discovery is to package the public keys used for token validation from our well-known token issuers with Cloud Connector backends directly instead of fetching them over the network, which currently requires 4 network calls to succeed whenever a server process expires its key cache.

This approach is demonstrated in PoC MR for AI Gateway.

Decision

We start with the incremental solution described above:

- Every Cloud Connector backend (currently: AI Gateway, Duo Workflow service; soon: GOB service, SAST service) will be packaged with public keys for both CustomersDot and gitlab.com, both for production and staging envs (4 keys total).
- Every Cloud Connector backend will load these keys from disk during application boot and hold them in process memory.
- We will start with the AI Gateway while continuing supporting OIDC discovery in parallel for other Cloud Connector backends (Duo Workflow service, GOB service, SAST service) and as a fallback during the transition.
- After confirming success with AI Gateway, we replace OIDC in every Cloud Connector backend and remove OIDC discovery for Cloud Connector authentication entirely.

Consequences

The proposed solution increases the stability of the Cloud Connector authentication:

- It eliminates CustomersDot and gitlab.com as a single point of failure for authenticating Cloud Connector features (both AI and non-AI).
- It removes the added complexity of OIDC discovery in our workflow.
- It eliminates network calls to OIDC endpoints when key caches expire in Cloud Connector backends.

Trade-offs:

- We will have to update all Cloud Connector backend service code repositories and redeploy them whenever we rotate keys.
  - Now, the system would always converge on the latest state but we still need to take cache into account which caused us incidents in the past.
  - We control all the systems that needs to be updated/redeployed so we can automate the rotation process in the future.
  - The planned key rotation is a very infrequent process. We currently do this once a year. Rather than that, it should only be done in case of the key leak.

Diagram: Current Architecture (OIDC key discovery)

Below is the flow we currently support for Self-Managed GitLab instances.
For gitlab.com, it is similar (gitlab.com acts as a Token Issuer instead of CustomersDot).

```
mermaid
sequenceDiagram
    %% Current Architecture (OIDC key discovery)
    Title: Current Architecture (OIDC key discovery)
    participant TokenIssuer as Token Issuer (CustomersDot)
    participant GitLabApp as GitLab Application
    participant Backend as Cloud Connector Backend

    TokenIssuer->>GitLabApp: JWT
```

```
    Backend->>TokenIssuer: OIDC discovery
    TokenIssuer->>Backend: JWKS
    GitLabApp->>Backend: Request with JWT
    Backend->>Backend: Validate JWT
```

Diagram: Proposed Architecture (bundled keys)

Below is the updated flow we suggest for Self-Managed GitLab instances.
For gitlab.com, it will be a similar change.

```
mermaid
sequenceDiagram
    %% Proposed Architecture (bundled keys)
    Title: Proposed Architecture (bundled keys)
    participant TokenIssuer as Token Issuer (CustomersDot)
    participant GitLabApp as GitLab Application
    participant Backend as Cloud Connector Backend

    TokenIssuer->>GitLabApp: JWT
    Backend->>Backend: Load JWKS from filesystem
    GitLabApp->>Backend: Request with JWT
    Backend->>Backend: Validate JWT
```

As you can see, we no longer need key fetches between the Token Issuer and the Cloud Connector Backend.

Key rotation changes

In case of key leaks or other security incidents, key rotation may be required.

With the described solution, the key rotation process will be different from what we use now.
For CustomersDot, the current key rotation process is described in issue 10387.

Currently, it works like this:

- First, we create a new key and add it to the published key set in the token issuer. This means the token issuer now publishes two different keys. We do this by having two environment variables that usually point to the same key, but diverge during a rotation.
- After that, we wait for Cloud Connector backends to perform OIDC key fetches (i.e. wait for key caches to expire; for AI GW, it is 24 hours) or rotate instance (which resets caches). They now have both keys in memory. This ensures that we can continue to validate tokens that were issued before the rotation.
- After 72 hours (token lifetime) of introducing the new key, we can point both environment variables to the same (new) key value.

If we remove OIDC, we have to change this sequence:

- First, we will create a new key pair and add the public key to the expected key set in all

Cloud Connector backend services. This prepares Cloud Connector backends to validate tokens signed with both the old and new key.
- We make sure that all Cloud Connector backends are redeployed and include the new public key.
- At this point, we can swap out private key in the token issuer.
- Finally, we can remove the old public key from all Cloud Connector backends.

We will document the key rotation process and link it to the Cloud Connector and related backends docsets.

Next steps

The approach suggested in this ADR allows us to make incremental improvements toward next solutions.
For example, in epic 14401 we explore the concept of self-contained JWTs, which would further simplify key management by embedding public keys directly in the tokens sent by clients. Since this requires validating the entire certificate chain, the key from which leaf certificates are derived must be available in backend services. Making this parent key available to backends could use the same approach we suggest in this ADR.

SECTION: engineering/architecture/design-documents/cloud_connector/decisions/003_unit_primitives.md

Context

Problems with the current Unit Primitives configuration

Service abstraction

The service abstraction was introduced to simplify client permission checks and centralize permission logic. By organizing features around "services," clients could determine if a user had access to a specific interface (for example, opening the GitLab Duo Chat window) without needing to understand underlying unit primitives or configurations.

Solved problems

Permission checks for UI elements

- Problem: Clients needed to check if the user had access to a UI element, not just the underlying feature.
- Solution: The service abstraction allowed clients to perform permission checks based on services like duochat or codesuggestions, without needing to delve into unit primitives.

Shielding clients from internal changes

- Problem: Internal backend changes, such as splitting or deprecating unit primitives, could disrupt client functionality.
- Solution: The service abstraction hid these internal changes from clients, ensuring that such changes didn't affect the interface presented to them.

Current problems

Despite its initial benefits, the services abstraction now presents several challenges.

- Mixed responsibilities:
  - The current interface mixes UI-related end-user permission checks with CloudConnector's domain responsibilities (instance-level access control).
- Configurable restrictions limited to service level:
  - All configurable restrictions (such as cut-off dates, minimum GitLab versions, and how unit primitives are bundled with add-ons or licenses) are defined only at the service level, not at the unit primitive level.
    - This prevents granular control over feature availability and packaging.
    - This means that services containing the same unit primitive can be configured with different cut-off-dates or bundled add-ons.
    - Example: documentationsearch is included in both the duochat and anthropicproxy services but with different configurations.
- Workarounds:
  - Special handling is needed for specific use cases, adding complexity to the system.
    - Example: anthropicproxy and selfhostedmodels services.
- Redundancy:
  -